Project: Smart Timetable & Resource Scheduling Management System
Technology: React.js, Node.js, PostgreSQL
AI Tools: Permitted

1. Project Objective

Develop a web-based Smart Timetable Management System for a college/university. The application should allow administrators to manage faculty, subjects, classrooms, student batches and generate weekly timetables while handling complex scheduling rules and resource conflicts.

2. Functional Requirements

A. Master Data Management

- Create, update, delete and view Faculty, Subjects, Classrooms and Student Batches.
- Maintain faculty availability, classroom capacity/type, batch strength and subject weekly lecture requirements.

B. Timetable Management

- Create, update and delete timetable entries.
- Assign faculty, subjects, classrooms, batches, days and time slots.
- Display weekly timetables with batch-wise and faculty-wise views.
- Support configurable working hours and lunch breaks.

C. Advanced Conflict Detection & Validations

The system must prevent:

1. Assigning the same faculty to multiple classes at overlapping times.
2. Booking the same classroom for multiple batches simultaneously.
3. Scheduling multiple subjects for the same batch at the same time.
4. Assigning faculty during unavailable hours.
5. Scheduling classes outside working hours or during lunch breaks.
6. Assigning classrooms with insufficient capacity.
7. Assigning practical/lab subjects to incompatible rooms.
8. Exceeding the weekly lecture limit of a subject.
9. Updating existing entries without revalidating conflicts.

D. Advanced Backend Requirements

- Implement database transactions and concurrency handling to prevent duplicate/conflicting bookings from simultaneous requests.
- Implement centralized error handling and meaningful API responses.
- Use proper database relationships, indexing and optimized queries.
- Implement authentication and role-based access (Admin/Viewer).
- Maintain basic audit logs for timetable modifications.

3. Automatic Timetable Generation – Advanced Challenge

Implement an automatic timetable generation feature that considers:

- Faculty availability.
- Subject-wise weekly lecture requirements.
- Classroom availability and capacity.
- Batch requirements.
- Working days and time slots.

The system should generate a valid timetable or clearly explain scheduling conflicts when no valid solution exists. Explain the algorithm and approach used.

4. Testing Requirements

Demonstrate:

- Faculty, classroom and batch scheduling conflicts.
- Invalid room capacity and room-type assignments.
- Lunch-break and faculty-availability restrictions.
- Weekly subject limit validation.
- Concurrent booking requests.
- Timetable update conflict validation.
- Successful and unsuccessful automatic timetable generation.

5. Technical Deliverables

- Complete frontend and backend source code.
- Database schema and sample data.
- REST API documentation (Swagger preferred).
- Basic automated test cases.
- README with setup and execution instructions.
- Working application demonstration.

Note: AI coding tools are permitted. However, the candidate must understand, validate and explain the submitted implementation, including architecture, business logic, database decisions and edge-case handling.

The focus is on engineering quality, problem-solving, scalability and correctness rather than simply completing the UI.